Messages in a POSIX message queue can be read one after the other by repeatedly calling *mq_read()*. Each *mq_read()* call gets only one message and to read the next message, the process must call *mq_read()* again, and multiple times thereafter to empty the message queue one by one.

In blocking mode, *mq_read()* gets unblocked when a message is received from the message queue. In non-blocking mode, *mq_read()* returns *EAGAIN* when no message is received, and non-zero positive value when a message is received. As mentioned earlier, receiving just a part of the message or a few bytes of a message is not possible.

Behavioural aspects of POSIX message queue

Creating/opening of message queues

mq_open() is the API to create a POSIX message queue or to open an existing POSIX message queue, with the first parameter being the name of the queue. It is the second parameter *oflag* that decides the behaviour of *mq_open()* API.

```
mqd_t mq_open(const char *name, int oflag);
mqd_t mq_open(const char *name, int oflag, mode_t mode,
struct mq_attr *attr);
```

To open an existing POSIX message queue, use *O_RDONLY* for read-only, *O_WRONLY* for write only, and *O_RDWR* for both send and receive. To create a non-existing POSIX message queue, use *O_CREAT*. To open in non-blocking mode, use *O_NONBLOCK*. If this is not used, then by default it would be a blocking queue. To check and create a non-existent new POSIX message queue, use *O_EXCL*. If the queue already exists, then it fails and returns EXIST.

If the message queue with the given name doesn't exist, *mq_open()* examines the third and fourth arguments to create a new POSIX message queue described by parameters *mode_t* and *mq_attr*.

Message queue descriptors

The file descriptor returns when any process calls *mq_open()*, always starts from 3 and increments thereon for every *mq_open()*. This is because Linux has reserved 0, 1 and 2 for *stdin*, *stdout*, and *stderr* respectively. For example, if a process A calls *mq_open()* to open/create a message queue named */mq_A*, 3 is returned as *mqd_t* if this is the first POSIX message queue opened by process A. At the same time, if another process, let us say process B, calls *mq_open()* to open/create a message queue name */mq_B*, here too 3 is returned as *mqd_t*, if this is the first POSIX message queue opened by process B.

The code given below demonstrates that for the same message queue name, *mq_open()* may return a different queue descriptor of type *mqd_t (int)*, for different (sender and receiver) processes.

```
// pmq_sender_desc.c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <mqueue.h>

#define QUEUE_PERMISSIONS 0660
#define MQ_MAX_MESSAGES 10
#define MQ_MAX_MSG_SIZE 128
#define MQ_MSG_BUFFER_SIZE MQ_MAX_MSG_SIZE + 10

int main(int argc, char **argv)
{
        mqd_t mqd_sender;
        struct mq_attr attr;
        const char *mq_name = "/mq-postman";
        char tx_buffer[MQ_MSG_BUFFER_SIZE] = {"Hello there
-  "};

        attr.mq_flags = 0;
        attr.mq_maxmsg = MQ_MAX_MESSAGES;
        attr.mq_msgsize = MQ_MAX_MSG_SIZE;
        attr.mq_curmsgs = 0;

        if((mqd_sender = mq_open(mq_name, O_WRONLY | O_
CREAT, QUEUE_PERMISSIONS, &attr)) == (mqd_t)-1)
        {
                perror("Client: mq_open");
                exit(1);
        }
        else
        {
                printf("Sender MQ %s opened with desc:
%d\n", mq_name,  mqd_sender);
        }

        for(int i=0;i<10;i++)
        {
                sprintf(&tx_buffer[14], "%d", i);
                printf("Sender: sending message: [%s]\n",
tx_buffer);
                // send message to message queue
                if(mq_send(mqd_sender, &tx_buffer[0],
strlen (tx_buffer), 0) == (mqd_t)-1)
                {
                        perror("Sender: Unable to send
```